

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
AI and Application Development and Jera

Task -2

Roll No.:T017	Name: Radhika mali
Class:TYIT	Batch:1
Date of Assignment: 29-7-2026	Date/Time of Submission: 29-7-2026

1. Design PEAS description for an Online Food Delivery Agent and classify its environment.

Code:

PEAS Description for an Online Food Delivery Agent

PEAS = {

```
"Performance Measure": [  
    "Fast delivery",  
    "Correct order delivered",  
    "Customer satisfaction",  
    "Minimum delivery cost",  
    "Timely delivery"
```

```
],
```

```
"Environment": [  
    "Customers",  
    "Restaurants",  
    "Roads",  
    "Traffic",  
    "Weather"
```

```
],
```

```
"Actuators": [  
    "Assign delivery person",  
    "Send notifications",  
    "Update order status",  
    "Recommend route"
```

```
],
```

```
"Sensors": [  
    "GPS",  
    "Customer order",  
    "Traffic data",  
    "Restaurant status",  
    "Payment confirmation"
```

```
]
```

```
}
```

```
print("PEAS Description")
```

```
for key, value in PEAS.items():
```

```
    print(f"\n{key}:")
```

```
    for item in value:
```

```
        print("-", item)
```

```
print("\nEnvironment Classification")
```

```
print("Observable      : Partially Observable")
```

```
print("Agents          : Multi-Agent")
```

```

print("Deterministic : Stochastic")
print("Episodic      : Sequential")
print("Dynamic        : Dynamic")
print("Discrete/Continuous : Continuous")
print("radhika")

```

Output:

```

PEAS Description

Performance Measure:
- Fast delivery
- Correct order delivered
- Customer satisfaction
- Minimum delivery cost
- Timely delivery

Environment:
- Customers
- Restaurants
- Roads
- Traffic
- Weather

Actuators:
- Assign delivery person
- Send notifications
- Update order status
- Recommend route

Sensors:
- GPS
- Customer order
...
Episodic      : Sequential
Dynamic        : Dynamic
Discrete/Continuous : Continuous
radhika

```

2. Implement Breadth-First Search to find the shortest path in a simple maze.

Code:

```

from collections import deque
maze = [
    [0, 1, 0, 0, 0],
    [0, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0]
]
start = (0, 0)
goal = (4, 4)
queue = deque([(start, [start])])
visited = {start}
directions = [(1,0), (-1,0), (0,1), (0,-1)]
while queue:
    (r, c), path = queue.popleft()
    if (r, c) == goal:
        print("Shortest Path:", path)
        print("Path Length:", len(path) - 1)

```

```

        break
    for dr, dc in directions:
        nr, nc = r + dr, c + dc
        if (0 <= nr < len(maze) and
            0 <= nc < len(maze[0]) and
            maze[nr][nc] == 0 and
            (nr, nc) not in visited):

            visited.add((nr, nc))
            queue.append(((nr, nc), path + [(nr, nc)]))
Print("radhika")

```

Output:

```

Shortest Path: [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (3, 2), (4, 2), (4, 3), (4, 4)]
Path Length: 8
radhika

```

3. Apply Bayes' Rule to calculate the probability of rain given cloudy weather.

Code:

```

P_Rain = 0.30
P_Cloudy_given_Rain = 0.80
P_Cloudy = 0.50
P_Rain_given_Cloudy = (
    P_Cloudy_given_Rain * P_Rain
) / P_Cloudy
print("Probability of Rain given Cloudy:",
      P_Rain_given_Cloudy)
Print("radhika")

```

Output:

```

Probability of Rain given Cloudy: 0.48
radhika

```

4. Solve the Missionaries and Cannibals problem using BFS or DFS.

Code:

```

from collections import deque
start = (3, 3, 1)
goal = (0, 0, 0)
# State = (missionaries_left, cannibals_left, boat_side)
# boat_side: 1 = left, 0 = right
moves = [
    (1, 0), (0, 1),
    (2, 0), (0, 2),
    (1, 1)
]
def valid(state):
    m, c, boat = state
    mr, cr = 3 - m, 3 - c
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False
    if m > 0 and m < c:

```

```

        return False
    if mr > 0 and mr < cr:
        return False
    return True
queue = deque([(start, [start])])
visited = {start}
while queue:
    state, path = queue.popleft()
    if state == goal:
        print("Solution Path:")
        for step in path:
            print(step)
        break
    m, c, boat = state
    for dm, dc in moves:
        if boat == 1:
            new_state = (m - dm, c - dc, 0)
        else:
            new_state = (m + dm, c + dc, 1)
        if valid(new_state) and new_state not in visited:
            visited.add(new_state)
            queue.append((new_state, path + [new_state]))
Print("radhika")

```

Output:

```

Solution Path:
(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
radhika

```

5. Implement A* Search for a graph and compare its result with UCS.

Code:

```

import heapq
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('A', 1), ('D', 2)],
    'C': [('A', 4), ('D', 1)],
    'D': [('B', 2), ('C', 1)]
}
heuristic = {
    'A': 3,
    'B': 2,
    'C': 1,
    'D': 0
}

```

```

}
def ucs(start, goal):
    pq = [(0, start, [start])]
    visited = set()
    while pq:
        cost, node, path = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        if node == goal:
            return path, cost
        for neighbour, edge_cost in graph[node]:
            if neighbour not in visited:
                heapq.heappush(
                    pq,
                    (cost + edge_cost,
                     neighbour,
                     path + [neighbour])
                )
def astar(start, goal):
    pq = [(heuristic[start], 0, start, [start])]
    visited = set()
    while pq:
        f, cost, node, path = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        if node == goal:
            return path, cost
        for neighbour, edge_cost in graph[node]:
            if neighbour not in visited:
                new_cost = cost + edge_cost
                new_f = new_cost + heuristic[neighbour]
                heapq.heappush(
                    pq,
                    (new_f, new_cost,
                     neighbour,
                     path + [neighbour])
                )
print("UCS Result:")
print(ucs('A', 'D'))
print("\nA* Result:")
print(astar('A', 'D'))
Print("radhika")

```

Output:

```

UCS Result:
(['A', 'B', 'D'], 3)

A* Result:
(['A', 'B', 'D'], 3)
radhika

```